

Programmazione 2

simi

October 2021

1 Logistica

Le lezioni cominciano alle 13.45 e finiscono quando ha voglia. **Per il laboratorio devo prenotare nel turno B1.** Per chi fa il laboratorio in remoto non lo cagano, comunque fanno partire la riunione su teams per cominciare l'esercizio e poi ce lo risolviamo da soli. Le domande vanno fatte poi in settimana su zulip o dove vogliamo.

2 Qualita' del software

La qualita' del software dipende da cosa si cerca, ci sono tipologie di qualita' che vanno a toccare ambiti molto diversi, ad esempio si differenziano qualita' del prodotto da qualita' del processo. Il processo tendenzialmente e' invisibile all'utente che non lo riguarda minimamente se non nell'output finale, ovvero la convinzione che un buon processo sviluppa un buon prodotto. Anche in processi interni non interessano molto all'utente (non guardera' mai il codice) in realta' si illude che non gli interessi, perche' ad esempio se richiede una modifica ed il codice che e' stato scritto non lo permette si freca. La nostra attenzione principale saranno le qualita' **interne al prodotto**. Ogni soluzione a un problema puo' essere rappresentata da un punto di uno spazio multidimensionale in cui ogni dimensione corrisponde a una qualita' del software. Spesso e' impossibile andare in due direzioni di qualita' contemporaneamente, anche il programmatore non riesce a decidere quale direzione di qualita' scegliere, per ovviare a questo problema e' importante confrontarsi durante la programmazione. Esiste il duck testing per spiegare all'anatra del bagno quello che stiamo facendo, perche' il solo cercare di spiegare qualcosa molto spesso ci dipana i dubbi. Non esiste in generale la soluzione, ma esistono diverse soluzioni anche difficilmente confrontabili tra di loro, wicked problem (sono problemi che per capire come va risolto esattamente lo capiamo dopo averlo risolto) nell'ambito dello sviluppo software e' amplificato dalle esigenze del cliente. Nel momento in cui mi pongo il problema di spiegare qualcosa a qualcuno ci sara' un momento in cui devo decidere come posso comunicare (le scelte di design). Il cliente spesso chiede una cosa che e' molto simile a quello che faceva prima della tecnologia perche' non conosce il mezzo (computer), viceversa l'informatico tende a dare troppe cose al cliente. Passiamo da questo approccio monolitico ad un approccio incrementale, quindi invece che raccogliere tutto subito raccolgo idee passo passo sviluppando un pezzettino alla volta rendendo partecipe il cliente dello sviluppo.

3 Refactoring

Il software ci condanna a dover accettare richieste di cambiamento in quanto e' impalpabile. Refactoring vuol dire aggiungere l'incrementabilita' anche mentre scriviamo codice, cominciamo a fare le cose nella maniera piu' semplice che ci viene in mente, funziona e pensiamo se possiamo fare meglio modificando la

nostra soluzione per cambiare le qualità interne del nostro prodotto. Esempi del perché dovremmo fare refactoring:

- Migliorare un design tenuto inizialmente "semplice" (o nato male)
- Preparare il design per una funzionalità che non si integra bene in quello esistente. Ristrutturare il codice (deve sempre funzionare) in modo che poi sia più semplice aggiungere una funzionalità che mi è richiesta o mi serve.
- Eliminare debolezze (debito tecnico). Delle volte accettiamo di lasciare qualche magagna nel codice e' definibile come debito tecnico, cioè significa che più tempo lasciamo qualcosa che non va nel codice più facciamo fatica a continuare a lavorare sul codice successivo (lavoro disagiatamente a causa del problema che ho lasciato prima) quindi e' come se pagassimo gli interessi sul nostro debito tecnico. A volte semplicemente non e' un codice così chiaro e se siamo in un team le persone che lavorano con me ci mettono il triplo del tempo a capire cos'ho fatto. Quindi devo non solo sapere quello che devo fare, ma devo anche spiegare il perché'.

4 Desing Knowledge

Dove tengo le scelte di design che ho fatto:

- **Memoria**, molto fallace e poco affidabile (difficile in team)
- **Documenti di design** (linguaggio naturale e diagrammi), il problema che sorge in questi casi e' il disallineamento in quanto tendenzialmente gli informatici sono cattivi scrittori ed odiano scrivere, quindi scrivono codice e si dimenticano di scrivere la documentazione
- **all'interno di piattaforme di discussione** (ad esempio forum), issue management, version control (sistemi di controllo delle versioni, git), però in generale non ci guida ad una versione organica ed ordinata, quindi risulta dispersiva e disorganizzata.
- **Con modelli specializzati** (UML e' un linguaggio nato per fare design del software). Può portare a approcci generative programming, tanto più sono formali questi linguaggi tanto più il passaggio tra programmazione e design tanto più può essere automatizzabile. O ci sono approcci round trip engineering vuol dire che posso modificare il modello (design) e mi modifica il codice e viceversa. Il problema e' che sono linguaggi difficili da usare e quindi si usano per progetti molto complessi.
- **nel codice**, ma spesso difficile rappresentare le ragioni (perché ho scelto di fare una scelta piuttosto che un'altra e' difficile da spiegare). Il codice dev'essere talmente chiaro che leggendo come leggo un romanzo devo capire quello che fa. Tipico dai progetti agile

A seconda del tempo, criticità etc posso scegliere approcci diversi.

5 Come condividiamo

Non il design specifico ma il know how:

- **Metodi**, insegno un metodo di lavoro ovvero lavoro sul processo
- **Design Pattern**, trovare delle situazioni di design talmente studiate di chiunque che sono stati scritti in modelli di progettazione (se hai questo problema una buona soluzione e'), diventa anche un metodo di comunicazione in quanto dicendo semplicemente un nome posso comunicare tutto un metodo di soluzione in maniera succinta e chiara.
- **Principi**(es. SOLID), segui questi principi e riuscirai a fare un software che ha certe qualita' (10 comandamenti). Ci sono talmente tanti principi che alcuni vanno in antitesi tra di loro, non c'e' un accordo ma ci dara' 5 principi che sono abbastanza riconosciuti nell'object orientation riconosciuti sotto l'acronimo SOLID

Se metto insieme queste tre cose le possibilita' di scrivere un bel codice aumentano notevolmente. Le prime difficolta' saranno di riuscire a scrivere del codice Java che compila. Guardiamo dev.java per i tutorial, ci consiglia anche di rifare degli esercizi di prog1 in Java.

6 Prova per ripassare

Le classi iniziano con maiuscolo come le costanti.

```
package it.unimi.di.p2.lez2;  
  
// i package vanno messi in directory, va messo un nome molto specifico  
  
public class Appuntamento{  
  
    // una classe in Java e' una struct in go, e' un tipo di dato che mi  
    // struttura una particolare struttura di dati  
  
    public int giorno;  
    int oraInizio, oraFine;  
  
    // il tipo delle variabili va prima del nome java le dichiarazione  
    // delle variabili non cambia la visibilita' se iniziano in maiuscolo  
    // o minuscolo lo facciamo con gli attributi private o public  
    // (visibile in tutte le classi)  
  
}
```

Java cerca di distinguere le tipologie di ciclo (quelli che so quanto vengono eseguiti: for; e il while cioe' cicla finche' e' vero quindi potenzialmente all'infinito).

7 Astrazione

E' una cosa fondamentale per lavorare in sistemi complessi. Nei nostri programmi abbiamo gia' una complessita' sufficiente per creare caos, in informatica e' molto facile creare entropia, ovvero creare caos all'interno del programma facendolo degenerare. Dovremo identificare e scomporre in sottosistemi (moduli, che caratterizzano l'object orientation). Non basta suddividere in pezzi il software, scopriremo che i concetti che spieghiamo sono molto semplici, i concetti dell'object orientation sono molto semplici, ma ci sono molte cose a cui prestare attenzione che non sono formalmente programmare ma saper programmare bene, per fare questo fondamentalmente bisogna cercare di capire tramite l'esperienza quello che sbagliamo o facciamo giusto, verificando nella pratica ci spieghera' le cose che servono e sono importanti. Bisogna capire quando un modulo e' buono e le due cose che classicamente si definiscono come qualita' richieste sono che i moduli dovrebbero avere una **coesione interna forte**(raggruppate), significa che devo mettere in un modulo cose che sono collegate tra di loro in maniera forte. Dentro un modulo di software ci devono essere cose che hanno legami forti. Dall'altra parte dev'esserci poco **disaccoppiamento esterno**(isolate), ovvero non devo lasciare legami esterni deboli, lasciare pochi fili sciolti. Non sara' possibile avere queste cose al 100% perche' ci saranno cose che non saremo certissimi in che moduli metterli.

7.1 Esempio

Mazzo da 52 carte:

- 4 semi
- 13 valori

La prima scelta e' quella di ideare una struttura dati (modello) con cui andare a definire le nostre variabili all'interno del programma. Deve rappresentare sia il mazzo sia una mano (sequenza) di carte tratte dal mazzo. Tipo le struct in go, inizialmente diciamo com'e' la struttura dati rappresentabile da quel concetto.

7.1.1 Astrazione come tipi

Mappiamo le 52 carte su 52 interi: [0:51] (0-12 Cuori, 13-25 Quadri, 26-38 Fiori, 39-51 Picche). usiamo il contenitore di interi per la sequenza di carte, dentro questo array metto degli interi. Quindi stiamo creando un'astrazione di una struttura dati nella maniera piu' semplice possibile,

```
class Deck {  
    int [] cards;  
}
```

```
//in go:  
type Deck struct{
```

```

    var cards [] int
}

```

L'idea di questa interpretazione e' solo nella nostra testa, quindi e' importante commentare. Non basterebbe una variabile di un tipo esistente? `int[] deck;` Perche' dobbiamo creare una struttura dati particolare e non usiamo direttamente un'array di interi. Creo un modello perche' ad esempio i miei dati potrebbero avere altre informazioni rilevanti e tramite la nostra classe possiamo aggiungerle agilmente e gestire. Inoltre piu' creo astrazioni piu' creo problemi, ad esempio in questo esempio usando una variabile gia' esistente potrebbero esserci tante carte tutti uguali. Quindi creare dei tipi ci da anche il modo di proibire delle cose che non dovrebbero succedere. Un altro problema potrebbe essere che se strutturassi semplicemente come un array di interi potrei sommarci qualsiasi altro intero, cosa che non voglio. Non sarebbe meglio anche definire un nuovo tipo per identificare la carta? ha senso se in qualche modo rendo pubblico il concetto di carta possono crearsi dei problemi, occhio ai concetti fondamentali dei nostri programmi, bisogna dargli una "dignita'"

7.2 Una carta

```

int card = 15 //15 = ??
int suit = card / 13 // 1 => Quadri -> ottengo il seme
int rank = card % 13 // 2 => tre (si comincia da zero) -> ottengo il valore

```

Ci sono tante informazioni che sono utili per definire la struttura dati, ad esempio ci sarebbe utile sapere quando una carta e' piu' grande dell'altra. Il problema e' che a volte strutturiamo le cose senza sapere esattamente i dettagli di quello che vogliamo. Se nascondiamo le nostre strutture dati possiamo cambiarle piu' facilmente, ad esempio la carta come intero e' scomodo in quanto e' scomodo dover fare operazioni per capire seme e valore e non inibisco operazioni non lecite, il fatto che non nascondo abbastanza e non rendo limitate le operazioni su una variabile rendo possibili delle operazioni illecite sulle variabili. Potremmo memorizzare seme e valore separatamente dentro ad un array, in cui il primo valore mi identifica il seme ed il secondo valore il valore della carta:

```

int [] card = {1, 3}; // 1 = Quadri, 3 = Tre (cominciando da 1)

```

Rimane ancora problematico, perche' in questo caso come rappresentiamo il mazzo? come array di array, risulta una matrice per cui ogni carta ha 52 valori. Possiamo rappresentare un oggetto complesso in diverse maniere, ci sono alcune piu' immediate da scrivere (array di interi) che pero' hanno certi difetti che per risolverli cerchiamo di astrarre il concetto. Creo una scatola a cui do un nome in cui poi non guardo piu' niente.

8 Information Hiding

Solo cio' che e' nascosto puo' essere cambiato liberamente senza pericoli.. I due scopi fondamentali dell'encapsulation sono:

- facilitare la comprensione del codice
- rendere piu' facile modificarne una parte senza fare danni

Astraggo alcuni passaggi (o strutture) in modo tale che queste cose siano prese per buone e poi sappiamo cosa farci (noi o i nostri colleghi). Quindi una volta che so il nome di una funzione possiamo riusarla senza dover andare ogni volta a vedere quello che fa e com'e' fatta. L'astrazione serve anche per ragionare sul codice in maniera piu' facile e modificarlo senza fare danni. Divide et impera. Devo non solo nascondere qualcosa ma anche aggiungere qualcosa: tipo una funzione `getRank` : dimmi il suo valore , quindi quelli che vogliono usare una carta sanno che devono chiamare la funzione `getRank` quindi a loro non interessa come noi abbiamo strutturato la carta. Nascondere non vuol dire rendere incomprensibile, significa rendere non accessibile all'altro ma molto chiaro a me. La sicurezza per offuscamento (windows) e' una sicurezza debole, tutti i protocolli di sicurezza seri sono open.

9 Problemi

Dobbiamo dare un significato chiaro a quello che facciamo. Mappare carte su interi non e' una buona idea:

- nel codice (a parte commenti), il significato di tali interi non e' chiaro
- facile dare valori impossibili (ad esempio il valore 100); non so cos'e' saltato fuori ma nulla dice al compilatore che 100 non e' valido. Dobbiamo definire un dominio di valori ammissibili
- Voglio cambiare rappresentazione, dovro' cambiare tutte le parti del codice. Se voglio cambiare rappresentazione di carta mi tocca cambiarla in tutto il codice che avro' scritto, cosa che non voglio.

10 Diamo un nome al tipo

```
class Card {
    int value; //0:51
}

class Deck {
    Card[] cards;
}

Card card = new Card();
card.value = 28;
```

Ci sono ancora un sacco di problemi, ma e' uno step intermedio per poterci lavorare. Abbiamo visto che e' inutile appiattirsi sui tipi base, sul fatto che l'astrazione e' importante, e sappiamo che una volta che proteggeremo e nasconderemo (dentro le classi) dobbiamo creare anche dei metodi di accesso. (dice di leggere anche il libro)

Riassuntino: come rappresentiamo l'informazione? come rappresentiamo le carte senza sapere cosa ci faro' successivamente. Ci eravamo accorti che un po' di chiarezza sarebbe potuta venire se creiamo dei tipi, nonostante dentro lasciassimo i valori primitivi (int). Se scrivo public e' pubblico a livello di package, quelle fuori e' privato, se metto private e' privato anche per le altre classi all'interno del package.

10.1 Definizione del dominio

Abbiamo preso un'espressione del dominio molto superiore a quello che ci serve, ci servono 52 interi mentre il tipo int ce ne fornisce miliardi. Si puo' definire un tipo citando esplicitamente tutti i valori che puo' avere, questo tipo si dice enumerativo che in Java e' **enum**, in questo caso se il compilatore trova un valore che non e' ammissibile ci dice che e' un errore

```
enum Suit {CLUBS, DIAMONDS, SPADES, HEARTS};  
//Cito esplicitamente tutti i valori  
enum Rank {ACE, TWO, THREE...};
```

```
class Card{  
    Suit suit;  
    Rank rank;  
}
```

```
Card card = new Card();  
card.suit = Suit.DIAMONDS;  
card.rank = Rank.THREE;
```

Da una parte rende il codice piu' leggibile e sono facili da usare ed efficienti, un po' verboso, nel momento in cui facciamo una print ci viene stampato direttamente la stringa corrispondente a quello che abbiamo scritto. Verranno istanziati solo al momento in cui sono veramente utili, viene creato l'oggetto la prima volta che ci serve e da quel momento in poi viene condiviso. Nello strutturare codice ed oggetti viene comodo creare oggetti immutabili, lo capiremo dopo. Pero' di nuovo per quanto riguarda la protezione non e' cambiato nulla, perche' dal mio corpo.

Deck poteva non cambiare ma guardiamo anche una struttura diversa (che usa i generics), ArrayList e' parametrico al contenuto dell'array:

```
class Deck {  
    ArrayList<Card> cards = new ArrayList  
}
```

Stiamo creando questa struttura e creiamo una carta assumendo subito dopo che a quella carta daremo valori validi, l'unico valore non valido in questo caso e' null, perche' cosi' come qualunque oggetto anche gli oggetti enumerativi assumono di default il valore null. Per poter risolvere questo problema dobbiamo introdurre il concetto di costruttore.

11 Costruttore

E' una particolare funzione della classe, ci possono essere diversi metodi chiamati costruttori di oggetti di quella classe. Se mi definisci un costruttore quello di default senza parametri non e' piu' lecito, quindi il null non e' piu' ammissibile. Posso comunque fregarlo esplicitando i valori di suit e rank il null. La cosa che posso fare e' limitare gli accessi alle classi. Non mi bastera' mettere a privato perche' sia utile, ma devo fornire dei metodi affinche' siano utili.

Una di queste funzioni e' il getter, creo delle funzioni nel metodo che mi permettono di accedere ai campi. Posso fornire un metodo per accedere al campo e non il set (ovvero non posso cambiare da fuori il campo). In questo caso abbiamo anche nascosto all'utente la struttura interna del Suit e del Deck perche' l'utente non sa che rappresentazione hanno, lui richiede solamente il valore. In realta' quando guardiamo una cosa ben progettata non ci sono i getter, perche' ho bypassato la protezione che avevo fatto.

12 Tell-Don't-Ask-Principle

Non chiedere i dati, ma di cosa vuoi che faccia sui dati. Invece di dire di darmi i valori dei campi singoli e poi li stampo io in maniera ammissibile, chiedi direttamente di stampare i campi:

```
public String toString() {return rank + " of " suit}
```

Arrivati a questo punto mi rendo conto che i getter non mi servono piu', perche' chi sta fuori dalla classe non gli serve sapere i valori di rank e suit. Vediamo un diagramma che rappresenta.

12.1 UML: Object Diagram

L'attributo e' un riferimento all'oggetto che viene puntato. Le due carte puntano allo stesso oggetto che ne definisce il seme, perche' hanno lo stesso seme, gli oggetti di tipo enumerativo vengono condivisi.

13 Escaping references

Ho reso privato qualcosa pero' non mi sono accorto che gli ho dato un'altra via per accedervi.

```

public class Deck {
    private List<Card> cards = new ArrayList();

    \\ Creo il deck di 52 carte
    public Deck(){
        for (Suit s : Suit.values())
            for (Rank r : Rank.values())
                cards.add(new Card(s, r));
    }

    @Override
    public String toString() {
        String ans = "There are " + cards.size() + " cards:\\n";
        for (Card currentCard : cards)
            ans += currentCard.toString() + '\\n';
        return ans;
    }

    public Card draw() {
        return cards.remove(new Random().nextInt(cards.size()));
        // Rimuovi la carta che peschi casualmente
        //tra 1 e 52 e la restituisce
    }
}

```

Aggiungiamo un getter per poterci fare un po' di cose :

```

public List<Card> getCards() {return cards;}

```

```

// ...

```

```

List<Card> myCards = myDeck.getCards();
myCards.add(new Card())

```

faccio un getter e rendo pubblico un riferimento ho dei problemi perche' ho esplicitato e reso modificabile qualche segreto della mia classe. Vogliamo dare delle responsabilita' di gestire gli elementi alla classe, la responsabilita' di gestire le carte e' del mazzo. Getter non e' un metodo sicuro per tirare fuori qualcosa di privato.

Se aggiungiamo un setter, e permetto di passarmi una lista di carte io ho di nuovo lo stesso diagramma, perche' ho un puntatore alla lista di carte. La stessa cosa con un costruttore. Perche' mi tengo un riferimento all'oggetto che gli ho passato. Una soluzione potrebbe essere passare tramite il getter un oggetto immutabile (nell'esempio la stringa e' un oggetto immutabile in Java).

Questo problema non crea un problema di compilazione, ma mi crea problemi di sicurezza. Se il main ha la possibilita' di modificare un elemento nell'oggetto privato e' un problema. Nell'esempio myCards punta alla lista di carte, alla

quale punta anche cards in Deck, che e' privato. Quindi l'utente puo' modificare la struttura del Deck.

13.1 Oggetti immutabili

Non c'e' maniera di cambiare lo stato dell'oggetto dopo la sua inizializzazione

- non fornire metodi che modificano l'oggetto
- rendi tutti gli attributi private
- rendi tutti gli attributi final, ovvero il valore viene dato una sola volta e poi non e' piu' modificabile
- assicura accesso esclusivo a tutte le parti non immutabili (no escaping references)

Non c'e' bisogno che tutti questi punti siano verificati, so per certo pero' che se tutti lo sono l'oggetto e' sicuramente immutabile. Usando oggetti immutabili posso fregarmene dell'escaping reference. Nel senso che comunque dal main possono vedere i dati che dovrebbero essere segreti, ma non possono modificarli. Comunque il fatto di poter leggere qualcosa non significa che sia meno grave di poterlo riscrivere.

Una card immutabile:

```
final class Card {
    private final Suit suit;
    private final Rank rank;
    public Card( Suit s, Rank r) { suit = s; rank = r;}
    @override public String toString() { return rank + " of " + suit; }
}
```

14 Exposing Internal Data

A volte puo' risultare necessario esporre parte dello stato interno. A volte e' necessario esporre i dati della classe. Nel nostro esempio il Deck verra' usato come mano, quindi il giocatore deve sapere che carte ha in mano, dobbiamo dargli quindi un modo per leggere i dati della classe. Non e' ammissibile il getter della lista di carte, come abbiamo visto l'altra volta ci crea problemi di possibile sovrascrittura dei dati dal main.

Aggiungere metodi che ritornano parti immutabili dello stato, o un loro derivato:

```
public int size() { return aCards.size(); }
public Card getcard(int pIndex) { return aCards.get(pIndex);}
```

Se invece di darti accesso direttamente ai dati, ti fornisco un metodo, che se chiamato ti da l'informazione che cerchi ma non ti fa vedere la struttura interna

alla classe. Questo metodo sicuramente e' sicuro, non do' l'accesso a tutta la lista di carte, ma ad una carta in una specifica posizione, perche' restituire la carta e' sicuro in quanto e' definita come struttura immutabile. Restituire il contenitore non lo e', in quanto e' una struttura modificabile. Componendo queste due funzioni e' possibile ottenere (iterare su) tutte le carte del mazzo:

```
for (int i = 0; i < myDeck.size(); i++)
    System.out.println(myDeck.getCard(i).toString());
```

Ciclo sul deck e chiamo i metodi creati prima per restituire la mano di carte senza dare accesso all'utente. Un altro metodo (volendo piu' banale) che puo' valere sempre che e' quello di copiare l'array list che contiene esattamente le stesse carte:

```
public List<card> getCards() {return new ArrayList<>(cards);}
```

Il costruttore ArrayList(collection) crea una nuova ArrayList a partire da una Collection e la inizializza con gli stessi elementi. Questo costruttore guarda quanti elementi ci sono in cards e li copia uno per uno in un altro ArrayList che li contiene, cosi' quando faccio il get non restituisco un puntatore a card ma restituisco una copia. Chiaramente restituire una copia e' qualcosa di dispendioso, dipende dalla quantita' di dati che devo passare. Qui stiamo facendo una shell of copy, non andiamo in profondita', ci accontentiamo di fare una copia del contenitore (arrayList) non gli oggetti stessi, in questo caso va bene perche' le carte sono immutabili, ma nel caso in cui lo fossero sarebbe un problema, perche' l'utente potrebbe cambiare la carte all'interno. Dovrei fare una deep copy:

```
public List<Card> getCards() {
    ArrayList<Card> result = new ArrayList<>();
    // per ognuna delle carte in cards aggiungi a result una nuova carta
    // che e' una copia di card
    for (Card card : cards) {
        result.add(new Card(card));
    }
    return result;
}
```

In questo caso non crea solo una copia dell'ArrayList, ma crea anche una copia di tutte le carte. Qui a maggior ragione c'e' un problema di efficienza.

Un altro metodo e' il **wrapper** fornito da Java della classe Collection che ci restituisce una ArrayList non modificabile, ti fornisce la stessa interfaccia della lista, pero' le operazioni che la modificano sono implementate per sollevare un eccezione. E' una cosa particolare pero' in certi ambiti puo' risultare che sia comodo.

Un altro metodo sarebbe usare degli **iteratori**. Ci torneremo per vedere in dettaglio come poterli fare. (Iterator = e' un pattern).

15 Design by Contract

E' un approccio alla progettazione.

```
public final class Card {
    private final Rank rank;
    private final Suit suit;
    public Card(Rank r, Suit s) {
        rend = r;
        suit = s;
    }
    public Rank getRank() {return rank;}
    public Suit getSuit() {return suit;}
}
```

Tutta questa roba e' sufficientemente protetta per stabilire che non puo' essere usata in maniera sbagliata? Ad esempio basta passare l'identificatore null e non abbiamo creato controlli su questo, quindi lo accetta:

```
Card card = new Card(null, Suit.CLUBS);
```

Troveremo dei modi per specificare meglio il valore null. In questo momento vorremmo vietare che una carta prenda il valore null. Chiameremo **Interfaccia** di una classe cio' che e' stato reso pubblico, in questo caso abbiamo reso pubblico i metodi e non l'implementazione. Non era neanche pensabile fare un controllo statico (cioe' che il compilatore li controlla sempre) con dei vincoli che potremmo volere. Percio' ci sono una serie di informazioni che sono legate alla semantica di uso della classe che noi vorremo andare a chiarire in modo efficiente e velocemente in modo da fissar ei confini tra chi crea la classe e chi li usa. In design contract ci sono dei contratti tra un "fornitore" (colui che scrive il metodo) ed il "cliente" (colui che invoca un metodo) che definiscono i diritti e obblighi. Il contratto fissa :

- Precondizioni: posso invocare il metodo solo se valgono delle condizioni. Sono predicati che devono essere veri alla invocazione del metodo. Come cliente posso invocare il metodo solo quando e' possibile chiamarlo. Ad esempio la funzione fattoriale che implemento puo' funzionare se l'intero che mi passi e' j 15, altrimenti non puoi chiamarla.
- Postcondizioni: ammettiamo che le precondizioni siano lecite, e' responsabile del programmatore che alla fine dell'esecuzione siano valide alcune cose. Dammi l'elemento i-esimo di una lista, precondizione: dev'esserci l'elemento i-esimo e la lista non e' vuota, allora il fornitore deve garantire che alla fine di questa cosa deve tornare esattamente l'elemento che ti ho chiesto. Modifichiamo il comportamento in termini di "se" "allora"

Se il cliente chiama il metodo quando non siano verificate le precondizioni qualunque cosa faccia quel metodo va bene. Se mi chiami quando non dovevi chiamarmi posso fare quello che voglio. Viceversa che mi hai chiamato in un

momento lecito (rispettando le precondizioni) e ti restituisco un valore che non rispetta le postcondizioni e' un problema mio. Se ho fissato questo contratto non mi devo preoccupare di controllare che l'utente inserisca dei valori corretti, quindi semplifica il lavoro del programmatore. Aver fissato dei limiti mi semplifica il lavoro di scrittura del codice. Come esprimiamo questi contratti:

```
/*
@pre pRank != null && pSuit != null
*/

public Card (Rank pRank, Suit pSuit) {
    rank = r;
    suit = s;
}
```

Robilland si e' creato l'annotazione @pre che rende strutturato un commento, dico all'interno del commento i valori che il cliente non deve mettere, indico quindi le precondizioni, quindi poi non devo piu' fregarmene di controllare che non ci sia null, perche' e' responsabilita' del cliente di rispettare queste precondizioni. Si possono mettere delle precondizioni ogni volta che si crea un metodo.

Si puo' fare con assert in Java, programmazione difensiva con asserzioni (opzione -ea):

```
public Card(Rank r, Suit s) {
    assert r != null && s != null ;
    //dopo condizione posso mettere stringa con asserzione se fallisce
    rank = r;
    suit = s;
}
```

In questo caso controllo le precondizioni e nel caso le precondizioni non sono soddisfatte sollevo un'eccezione e faccio saltare tutto. Non sono vincolato a controllare le precondizioni, perche' assumo che siano verificate, se io le controllo (programmazione difensiva) faccio una soluzione meno efficiente del programma. Assert e' un buon modo perche' durante lo sviluppo ci sono delle fasi in cui non sono sicuro che tutto sia chiamato bene quindi ha senso che me lo controlli, una volta che e' finito posso togliere l'asserzione perche' tanto so che va bene sempre quando metto dati giusti. Devo compilare il programma con l'opzione -ea perche' altrimenti mi tratta l'assert come un commento. Potrei mettere un if al posto di assert ma :

- la faccio sempre
- sollevo un'eccezione che e' diverso da quello che succede con assert

Non e' necessario mettere gli assert, ad esempio se devo tornare la somma degli elementi di una lista e metto come contratto il fatto che la lista non dev'essere vuota se lo e' puo' succedere tutto.

(e' il capitolo 3 del libro) Il lavoro del design e' trovare la collaborazione tra le classi. Cerchiamo ora di estendere il concetto di tipo. E' caratteristica dei linguaggi OOP la definizione delle classi puo' non partire da zero, ma puo' partire per inferenza di una classe gia' nota. Ho gia' definito una classe e definisco una nuova carta dicendo che e' come quella ma con una conoscenza in piu'. Questo discorso di definire per differenza ci permette di semplificare la scrittura, ma se fosse solo l'ereditarieta' per implementativa sarebbe una cosa molto banale. In Java l'ereditarieta' e' fatta per creare il concetto di sottotipo, e' come quello (un quadrupede puo' essere sia un cane che una mucca), questa cosa si chiama **POLIMORFISMO**, ovvero puo' prendere due forme. Il nostro `ArrayList` puo' essere visto come lista semplice, come una serie di oggetti ordinati. Dall'altra parte posso dire che una variabile implementata come `collection` puo' puntare ad un `array`, `ArrayList`, o una `collection`. L'ereditarieta' ha una direzione (gli oggetti piu' astratti possono puntare ad oggetti piu' concreti). Ha un ambito piu' semplice se invece che parlare di generalizzazione parliamo di interfacce e classi.

16 Interfaccia

Ha un significato in astratto, uno in Java e quando applicato in Java evolve (differenze tra Java5 all'8). E' l'insieme delle operazioni accessibili alle altre classi, non qualunque cosa pubblica ma solo i metodi pubblici, dando per scontato che non ci possono essere altri modi di interagire con una classe se non tramite i metodi (gli attributi tutti privati). Possono esserci eccezioni per cui metto attributi `public`, ma di prima istanza quando creo una classe devo mettere gli attributi `private`. Introduciamo il concetto di Cliente delle classi, ovvero chi vuole usare i metodi della classe.

```
public class Deck {
    public void shuffle() {}
    public Card draw() {}
    public boolean isEmpty() {}
    public void sort() {}
}

private class Client {
    private Deck deck = new Deck();
    ...
    deck.shuffle();
}
```

Le due classi sono fortemente accoppiate: `Client` conosce `Deck` e puo' lavorare solo con quello. E' vincolato a quella specifica tipologia di classe (oggetto). Questa cosa ci puzza per un buon design.

17 SOLID principles

- single responsibility: una classe dovrebbe occuparsi di una cosa sola. Dovrebbe avere tutto quello che gli serve per fare il suo compito e nulla di più. (in una situazione ideale). L'ereditarietà ci direbbe che ci sono cose in comuni tra le classi.
- open/closed: pensare già che quando vorrò aggiungere qualcosa non devo andare a ritoccare il codice della classe esistente. Potrei rovinare anche la cosa vecchia.
- liskov substitution: sostituibilità, posso sostituire un oggetto con l'altro a partire da ereditarietà
- interface segregation
- dependency inversion: (client che dipenda da dack), una classe concreta dipenda a una altra classe concreta. Cio' che e' di alto livello non deve dipendere da cosa di basso livello, entrambi dovrebbero dipendere dall'astrazione.

17.1 Estraiamo le interfacce

Facciamo una funzione che ci permette di pescare n carte, la andiamo a basare su quella che e' l'interfaccia della classe Deck.

```
public static List<Card> drawCards(Deck deck, int number) {  
    List<Card> result = new ArrayList<>();  
    for (int i = 0; i < number && !deck.isEmpty(); i++) {  
        result.add(deck.draw());  
    }  
    return result;  
}
```

Probabilmente abbiamo già definito un contratto con la draw, dicendo che dobbiamo preoccuparci di chiamarla solo se il mazzo non e' vuoto.

18 Interfaccia e polimorfismo

Le interfacce sono una serie di metodi che sono per forza pubblici. Cio' che verrà reso noto, non do' implementazione, semplicemente dico che sa fare quella roba lì:

```
public interface CardSource {  
    /*  
    @return The next available card.  
    @pre !isEmpty()  
    */  
}
```

```

        Card draw();

        /*
        @return True if there is no card in the source
        */
        boolean isEmpty();
    }

    public class Deck implements CardSource {...}

```

Dentro la dichiarazione della classe Deck aggiungiamo l'esplicitazione del rapporto che c'è tra la classe e l'interfaccia. Deck implementa CardSource. Il compilatore controlla che la classe deve saper rispondere al comando draw ed isEmpty. lo scopo è creare un parametro e dire che la mia richiesta su quel parametro è più generica. Rendo generica una funzione per potergli passare cose, chiunque implementi le funzioni della interface può essere passato ad un metodo.

18.1 Collegamento dinamico

Il compilatore non sa a tempo di compilazione quale metodo draw() dovrà chiamare. Rimanda la decisione al momento effettivo della esecuzione.

```

public static List<Card> drawCards(CardSource deck, int number) {
    List<Card> result = new ArrayList<>();
    for (int i = 0; i < number && !deck.isEmpty(); i++) {
        result.add(deck.draw());
    }
    return result;
}

```

Permette di chiamare codice non ancora scritto. Se in futuro voglio aggiungere funzionalità che ha una funzione che draw(), la mia interfaccia lo sa già chiamare, non ho bisogno di modificare il codice.

18.2 Esempio libreria Java

```

public class Deck {
    private List<Card> cards = new ArrayList<>();

    public void shuffle() {Collections.shuffle(cards);}
}

```

- cards è un oggetto di tipo ArrayList
- che implemente l'interfaccia List
- Che è passabile come parametro a shuffle di Collections

- visto che accetta List di qualunque elemento

Collections e' una classe dell'interfaccia List che ha un metodo shuffle a cui passo cards. Per fare la sort dobbiamo definire il concetto di oggetto confrontabile, ci fa venire in mente un'interfaccia:

```
public interface Comparable<T> {
    public int compareTo(T o);
}
```

Definisce un singolo metodo che restituisce un valore minore, uguale o maggiore a 0 a seconda del risultato del confronto. Questo metodo e' quello che usera' la sort di Collections. Quindi la interfaccia di Collections.sort e':

```
public static <T extends Comparable<? super T>> void sort(List<T> list):
```

Se T implementa una lista dell'interfaccia Comparable di T allora posso ordinare la lista perche' tutti gli elementi che la compongono sono confrontabili.

Ordinamento:

```
class Card implements Comparable<Card> {
    private CardOrderType orderType;

    public int compareTo(Card other) {
        switch (orderType) {
            case SUITFIRST :
                return suit.compareTo(other.suit);
            case RANKFIRST:
                return rank.compareTo(other.rank);
        }
    }
}
```

Questa cosa dello switch case e' un antipattern (si e' scoperto che e' un'implementazione scomoda).

19 Oggetto funzione

Esiste il concetto di Comparator che ci dice se gli oggetti sono comparabili:

```
public interface Comparator<T> {
    int compare(T o1, T o2);
    ...
}
```

```
//usata da
Collections.sort(List<T> list, Comparator<? super T> c)
```

In Java c'e' l'overloading dei nomi dei metodi, posso sovraccaricare di significato un nome, possiamo dare lo stesso nome a dei metodi, sappiamo la differenza

guardando i parametri. Ho fatto solo un'interfaccia, devo creare delle classi che implementano l'interfaccia:

- Posso usare classi innestate: tra le cose che posso mettere all'interno delle classi posso metterci definizioni di altre classi, creando un legame speciale tra la classe interna e quella esterna, ad esempio quella interna può vedere gli argomenti privati della esterna (perché è tra le graffe).
- Posso usare classi anonime: sono un particolare tipo di classi innestate
- Posso usare lambda expression: sono (per quanto ci riguarda) un po' di zucchero sintattico, diventano più facile da scrivere.

20 Classi innestate

Risolve il problema che il confronto sarà probabilmente basato su campi privati non più accessibili. Le classi innestate possono accedere ai campi privati della classe che li contiene:

```
public class Card {
    private Suit suit;

    public static class SuitComparator implements Comparator<Card> {
        @Override
        public int compare(Card card1, Card card2) {
            return card1.suit.compareTo(card2.suit);
        }
    }
}
```

```
Collections.sort
```

Lo abbiamo messo dentro la classe carta, quindi se dovessi creare un'altro comparatore dovrei cambiare la struttura della carta. Ciò nonostante non ho dovuto rilassare l'encapsulation.

21 Laboratorio

Non possiamo creare variabili a partire da una interfaccia. (Prima `a = new Prima()` dove `Prima` è un'interfaccia non si può fare, al contrario `Seconda b = new due()` dove `Seconda` è un'interfaccia e `due` è un oggetto si può fare) così posso usare solo i metodi dell'oggetto `b` che ho dichiarato in `Seconda`, non tutti quelli che ho in `due`.

22 Iteratori

Li avevamo già citati come mezzo per dare accesso a parti private proteggendo encapsulation. Rispetto alla soluzione che fornisce una copia della lista ha vantaggi che:

- Nasconde dettagli sulla struttura contenitrice
- La struttura risulta di default read-only

L'astrazione avviene tramite le funzioni `hasNext()` e `next()`. Con il `next` si inizia sequenzialmente a leggere tutti gli elementi, `hasNext` mi dice se c'è un prossimo. Dobbiamo quindi rendere una classe iterabile utilizzando un'interfaccia `Iterable`, per fare questo devo implementare il metodo `iterator`, basta scrivere su IntelliJ `nomeclasse implements Iterable<nomeLista di oggetti su cui vogliamo iterare>`, e lui ce lo crea in automatico:

```
public class PokerHand implements Iterable<Card>{

    List<Card> banco = new ArrayList<>();

    //per rendere itrabile la classe PokerHand creo il metodo Iterator ,
    // per farlo basta scrivere implements Iterable<Card>
    // e IntelliJ lo fa in automatico. Cambia se Card e' una lista o un array
    @Override
    public Iterator<Card> iterator() {
        return banco.iterator();
    }
}
```

Facendo questa cosa poi quando devo ciclare sugli elementi di `Card` potrò usare il comodissimo `for each` (creo una variabile `card` di tipo `Card` con cui poi ciclo all'interno di `banco` che è una lista di `Card`):

```
for (Card card: banco) {
    //istruzioni ...
}
```

Ho anche i metodi `hasNext()` e `next()`.

23 Campo `SUIT_ORDER`

Possiamo avere un unico esemplare di un oggetto interfaccia, così lo usiamo dove abbiamo bisogno senza doverne creare uno ogni volta. Creiamo un campo statico così lo riusciamo.

```
public class Card {
    public static final Comparator<Card> SUIT_ORDER = new SuitComparator();
    private Suit suit;
```

```

        private static class SuitComparator implements Comparator<Card>;
        @Override
        public int compare (Card card1, Card card2) {
            return card1.suit.compareTo(card2.suit);
        }
    }

```

String ha piu' di un tipo di ordinamento sensato, e' un oggetto comparabol ha il metodo `compareTo()`, ha al suo interno un campo che definisce un comparatore senza tenere conto delle maiuscole o minuscole. Posso farlo anche tramite funzioni lambda, e; figo perche' oltre alla `comparing` ho altre funzioni chiamabili ad esempio `thenComparing` (a partia' di questo guarda quest'altro).

24 Diagramma delle classi

Possiamo andare a ragionare sulle astrazioni che gli oggetti vanno ad implementare, non per forza a runtime, lo chiameremo diagramma delle classi. Graficamente e' simile a quello delle classi, le scatole sono classi ed interfacce (non oggetti), in ognuna di queste cose dico se e' un'interfaccia o una classe, abbiamo una freccia tratteggiata con una freccia bianca, e' `implements` (una classe implementa un'interfaccia). L'interfaccia e' caratterizzata da un'elenco di metodi, gli elenchi (tutti i metodi nell'interfaccia sono pubblici, non indico neanche che sono pubblici lo do per scontato). Un'interfaccia puo' estendere (`extend`) un'altra interfaccia, per cui quando una classe implementa un'interfaccia non solo si prende carico di implementare i metodi che l'interfaccia dice ma anche quelli delle interfacce che l'interfaccia estende. Se e' molto lunga la catena di generalizzazione diventa complicato capire cosa sta succedendo, quindi evitare di creare catene lunghissime. Interfacce hanno solo metodi, nelle classi oltre ai metodi posso avere anche attributi (indicati nella seconda zona del box), sia i metodi che attributi bisogna specificare qual'e' lo scope di visibilita' (`private`, `public`, `protected`, `package` (- per privato + per pubblico)), se ci sono dei costruttori con almeno un parametro allora vanno espressi, se c'e' solo quello senza parametri si puo' omettere, perche' si da per scontato. La freccia continua con la punta aperta e l'altra con il rombo all'origine sono delle frecce che esprimono relazioni tra istanze delle classi, guarda che' un'istanza di questa classe conosce (ha tra le sue responsabilita' e che lo definisce, un attributo) un oggetto di tipo. sono frecce di associazione. Posso esprimere con quanti oggetti dall'altra parte mi posso collegare (con un intervallo o con un numero preciso) si chiama cardinalita'. La differenza tra associazione ed aggregazione e' fine (in entrambi i casi sono implementate tramite un'attributo) nel caso di relazione dico che sono in relazione (un professore e' in relazione con gli studenti), con l'aggregazione sto dicendo che quello con cui sono in aggregazione mi definisce (un corso e' in aggregazione con degli studenti, perche' il corso esiste solo se ci sono studenti). L'ultima freccia e' quella della dipendenza (tratteggiata ed aperta) quando scrivo il codice di questa classe posso dover tener conto di quello che e' stato scritto nell'altra classe (dipendo da com'e' stata scritta la classe

prima) ho due classi una scrive un file in un formato e l'altra legge lo stesso file, se cambio com'è stato scritto il file devo cambiare anche quella che la legge, quindi le classi sono in relazione ma a runtime non hanno nessuna relazione. Le dipendenze non sono sempre cose bellissime, potrebbero non essere visibili (me ne dimentico), potrebbe essere più sensato andare ad esplicitare le dipendenze, nei file c'è dire esiste una classe che legge e scrive il file in quel formato, la prima la aggrega e usa la scrittura, la seconda la aggrega e usa la lettura. Una classe deve avere un'unica ragione per cambiare (non deve avere tanti scopi una classe).

24.1 Esempio

Quello del comparable, c'è un interfaccia comparable che può essere implementata da un oggetto, card è un oggetto comparable ed essendo comparable deve definire un metodo compareTo(T). Lo faccio perché c'è una classe Deck che aggrega da 0-52 carte, le istanze della classe deck dipendono dalla classe collection che definisce un metodo statico sort (tra le altre cose) e questa collection assume che T sia qualcosa che estende Comparable<T>, cioè che sia una lista di oggetti comparabili, quindi di nuovo il sort dipende da com'è fatto comparable. Rank e Suit potrebbero anche essere messi come attributi della classe Card.

24.2 Iterator Pattern

Provvede all'accesso degli elementi aggregati di un oggetto sequenzialmente senza esporre la rappresentazione interna.

24.3 Strategy Pattern

Ho qualcuno che vuole fare qualcosa ma può essere fatto in diverse maniere. Ho il sort che vuole ordinare e quello che gli serve è avere una strategia di confronto, nel momento in cui lo sa può definire qualsiasi algoritmo vuole.

25 Chain Responsibility

Definisco dei controllori separati uno per ogni associazione e chiedo a quello di valore più alto e se la risposta è sì la ritorno se non è lui prova con il prossimo.

26 Nullability

La not null possiamo metterla su un parametro come argomento di una funzione, quando la viene chiamata ci aspettiamo che non venga chiamata con un valore null. Se invece il null fosse un valore ammissibile? tipo se aggiungessimo il joker all'interno delle carte, i getter potrebbero ritornare:

- null, ma e' sconsigliabile
- un valore qualsiasi (2 di picche): tanto non e' considerato se isJoker == true, ma porterebbe a confusione
- aggiungo un valore a enumerativo: ma cosi' ci sarebbero 5 segni e 14 rank

tramite il polimorfismo si possono adattare certi metodi in base al caso, tramite un interfaccia potremmo dire che esiste un oggetto nullo se vogliamo caratterizzarli in maniera speciale:

27 Pattern: NullObject

Vogliamo creare un oggetto che corrisponda al concetto di "nessun valore", l'oggetto neutro, per ogni metodo che vado a definire ricso a crearmi un implementazione nel caso in cui l'oggetto non ci sia.

```
public interface CardSource {  
    //creiamo un oggetto di una classe nulla  
    public static CardSource NULL = new CardSource() {  
        public boolean isEmpty() {return true;}  
        //non entrera' mai al return null perche' se chiamo una draw su un  
        //oggetto null mi torna sempre un errore  
        public Card draw() {  
            assert !isEmpty();  
            return null;  
        }  
        public boolean isNull() {return true;}  
    }  
    Card draw();  
    boolean isEmpty();  
    default boolean isNull() {return false;}  
}
```

Il metodo di default e' dare un implementazione dentro all'interfaccia, se la mia classe che implementa cardSource non la ridefinisce la eredita, se la prende gia' fatta. Dentro l'interfaccia ci possono anche essere degli attributi, ma che devono essere pubblici, statici (non a livello di oggetto ma a livello di classe, NULL e' un attributo che viene allocato ed istanziato non quando viene creato l'oggetto ma quando istanzio la classe, esiste prima che creo l'oggetto). Tutte

le classi hanno visibilit  di un primo oggetto di CardSource che chiamiamo NULL che definiamo il loco (nell'interfaccia) con una classe anonima che ritorna is empty, draw e nel suo caso isNull e' true. CardSource.NULL e' un oggetto valido di un tipo anonimo che aderisce alla interfaccia CardSource ma che ha particolari implementazioni, ma e' una cosa da fare poco perche' l'idea che c'e' sotto e' di trattare un oggetto NULL come qualsiasi altro oggetto. Cerchiamo di assimilare gli attributi nelle interfacce perche' le useremo. Un altro modo che e' stato introdotto in Java piu' moderni e' i tipi Optional:

28 Optional

E' inteso per risolvere i problemi di tornare un tipo che non so se sto tornando veramente il valore o l'errore. Una delle peculiarita' e' che non ha un costruttore, ma ci sono metodi statici per la creazione:

- static Optional empty(): creo un oggetto che avra' valore nullo e sara' identificato come nullo
- static Optional of(T value): dato un tipo di da un optional di quel tipo, assumo che valore sia non nullo e restituisco l'optional di quel valore non nullo
- static Optional ofNullable(T value): restituisce un optional che ritorna empty o of di T a seconda se value e' nullo o no

Fornisce dei metodi:

- T get(): se lo chiamiamo su un empty ritorna un eccezione, se invece non e' nullo ritorna il valore
- boolean isEmpty()
- boolean isPresent(): se il valore e' presente torna true
- T orElse(T): se il tipo optional aveva un valore ritorna quel valore se era empty torna quello che ti do io
- void ifPresent(Consumer): value.ifPresent(valore - System.out.println(valore.lenght()));

29 Final

Gli attributi, variabili o parametri (lo si puo' mettere anche su una classe, ad esempio la classe String). Ha una grossa importanza quando lo si usa sugli attributi di una classe, possono essere assegnati solo una volta (nel costruttore o come parte della dichiarazione). Non sono delle costanti, tranne nel caso in cui gli oggetti di tipo final sono immutabili.

30 Identity, Equality, Uniqueness

Sono tre concetti legati tra di loro ed hanno un grandissimo impatto pratico :

- Identita': sono lo stesso oggetto. Viene valutata nel caso degli oggetti tramite l'operatore `==`.
- Uguaglianza: rappresentano uno stesso valore, ma sono oggetti diversi. Dovrebbe essere valutata tramite il metodo `equals(Object)`, che pero' di default la rimappa su identita', fa un confronto tra i riferimenti. Non e' possibile che java ci dia una rappresentazione di `equals` che ci vada bene sempre, siamo noi delegati a fare questa rappresentazione.
- Unicit : quando identita' ed uguaglianza coincidono, per ogni possibile combinazione di valori ho un unico oggetto. Cioe' non possono esistere due oggetti distinti che sono considerati uguali. String ha questa caratteristica

L'unicit  se la ottengo oltre ad essere richiesta in alcuni casi porta comunque diversi vantaggi sia di prestazioni sia di chiarezza e leggibilit . Ci sono due pattern legati al concetto di unicit : Singleton (era usato in maniera sbagliata ma in certi casi e' utile, il concetto di avere nativamente nel modo piu' semplice possibile il concetto di oggetto invece che di classe con cui istanzio oggetti, ho solo un oggetto e tutti useranno quello), FlyWeight.

31 SINGLETON

Invece di classe vorrei avere quello di un singolo oggetto. Cioe' una classe che puo' avere solo un istanza.

- Non puo' esserci un costruttore pubblico
- Devo prevedere un metodo statico per farmi dare l'unica istanza (`getInstance()`), questo metodo per essere invocato prima della prima istanza dev'essere statico
- Un attributo statico dove mettere l'unica istanza

```
public class Singleton {
    private static Singleton instance;
    //dentro il costruttore ovviamente posso fare tutte le cose
    // che faccio sempre con i costruttori
    private Singleton() {}
    public static Singleton getInstance() {
        if (instance == null) instance = new Singleton();
        return instance;
    };
    public void sampleOp() {...};
}
```

E' implementazione non thread safe, se viene tolto il controllo esattamente dopo aver controllato che `instance == null` e lo passa al thread successivo che fa il controllo e crea un oggetto, poi il controllo torna al thread 1 che avendo fatto il controllo si crea un nuovo oggetto Singleton, mandando a puttane tutto il ragionamento di prima. Ovviamente se poi vado a creare un nuovo oggetto Singleton andra' ad implementare esattamente lo stesso oggetto.

In Java c'e' un idiomma che rende molto facile la creazione di un Singleton:

```
public enum Singleton {  
    INSTANCE;  
    public void sampleOp() {...}  
}
```

Sfrutta il fatto che ai vari colori che poteva avere un enumerativo corrisponde un oggetto, se gli do' un unico valore il risultato e' che ho un unico oggetto per l'enumerativo.

```
public enum Singleton {  
    INSTANCE;  
    private final String segreto = "pippo";  
    public void sampleOp() {...}  
}
```

Nel main quando devo fare l'oggetto chiamo semplicemente:

```
Singleton pluto = INSTANCE;
```

32 Equals e hashCode

Bisogna sempre fare l'override di `hashCode` quando lo faccio per `equals`. `hashCode` e' una funzione sono delle funzioni non invertibili che dato un contenuto ti danno una serie di bit che dipendono da quel contenuto, danno stringhe rappresentanti il contenuto ma piu' facili da rappresentare. Ma e' sufficientemente standard da avere una costruzione semiautomatica in IntelliJ, ricordati che genera in automatico gli override di `equals` e `hashCode`

33 FlyWeight

E' un pattern legato al concetto di unicit  ed immutabilit . Quando le istanze equivalenti sono fortemente condivise all'interno del programma diventano auspicabili sia immutabilit  che unicit . Ad esempio le nostre Card. Serve a gestire una collezione di oggetti immutabili assicurandone l'unicit , quando li creano non possono cambiare, ma per ogni stato concreto di questi oggetti voglio un unico esemplare, in modo che l'unicit  implichi l'uguaglianza e viceversa. Un esempio molto calzante per noi e' la carta che e' una delle classi usate di piu', all'inizio ci siamo stupiti che non c'era un costruttore per la carta ma solo la possibilit  di averla (`get`), quindi delego alla classe di creare gli oggetti (come

fatto con il singleton, get instance non new instance) la stessa cosa con il fly-weight solo che non ho solo un oggetto ma piu' di uno. Potrei creare tutte le carte all'inizio oppure creare l'unica carta alla sua prima istanza:

```
public class Card {
    private static final Card[][] CARDS =
        new Card[Suit.values().length][Rank.values().length];

    static {
        for( Suit suit : Suit.values() ) {
            for (Rank rank : Rank.values() ) {
                //vado a riempire l'array con le carte in ordine
                CARDS[suit.ordinal()][rank.ordinal()] = new Card(rank, suit);
                //il costruttore che non vedimao e' privato, quindi solo qui
                posso usarlo
            }
        }
    }

    public static Card get(Rank prank, Suit psuit) {
        assert prank != null && psuit != null;
        return CARDS[psuit.ordinal()][prank.ordinal()];
    }
}
```

L'array di array di card potrei crearlo dentro i costruttori oppure direttamente in linea alla sua dichiarazione, a questo punto l'assegnamento non potrà piu' comparire neanche nel costruttore, lo posso mettere lì se per tutti i costruttori che poi definisco non lo riassegno piu'. Ed e' questo array che e' immutabile in cui metto tutte le carte. Ho bisogno di fare anche qualcosa altro, tendenzialmente dobbiamo creare le variabili nel costruttore, ma questo attributo e' statico (esiste prima del costruttore), in java esiste il costrutto (raro) blocco di inizializzazione statico che e' una parte di inizializzazione degli elementi statici, lì dentro metto una serie di istruzioni che vengono chiamate quando creo la classe, non gli oggetti della classe. Non e' vero che il programma inizia con la prima istruzione del main ma ci sono altre operazioni prima, tipo l'inizializzazione degli oggetti statici ed e' un casino se il programma non funziona prima di fare il main perche' ci sono dei problemi delle cose statiche. La nostra parte statica sceglie di creare tutti gli oggetti come prima cosa, per poi semplificare l'operazione del get perche' e' semplicemente restituiscimelo, non devo preoccuparmi di controllare se l'oggetto ho creato. Se invece creassi le carte alla prima istanza sarebbe un approccio algoritmico lazy, ovvero creo una cosa solo quando mi serve per evitare di sprecare tempo a creare una roba che poi potrei non usare mai. Ricordiamo che con il get ci torna l'unico esemplare di quella carta con quel valore che ci sara' per sempre, la carta e' immutabile quindi posso restituirla in piu' posizioni. Quando parliamo di stato c'e' ancora un'ultima cosa che e' avanza, ma capita di trovare.

34 Nested Class

Innestare una classe in un'altra classe e' difficile e tutto sommato i vantaggi si scoprono solo per cose tanto fini. Si dividono in vari tipi:

- Statiche
- Non statiche (Inner class) divise in :
 - Locali
 - Anonime

Quelle piu' comuni sono le anonymous class (non statiche innestate dentro altre classi), sono delle classi a cui non diamo un nome per cui definiamo quando ci serve una loro istanza (tipi comparator). L'altra da usare e' la static nested class pero' sono molto diverse perche' questa e' static.

Da questo schema sembrerebbe che le classi anonime siano di tipo non statico e sembrerebbe esserci un riferimento alla classe anonima che la crea

34.1 Static nested classes

Posso creare

34.2 Anonymous Inner Class

Ho un attributo che mi collega alla classe che mi contiene ma la maggior parte delle volte non mi serve e non lo cito neanche come nel comparator, Java non prevede una classe anonima static quindi sembrerebbe di avere sempre questo oggetto, ma il compilatore Java permette di creare un'istanza della classe anonima all'interno di un metodo statico. E' molto utile poterlo fare pero' significa che lo schema di prima non e' vero ce la prendiamo cosi' e bona.

34.3 Anonymous Inner Classes... static?

Crea un comparatore in maniera anonima e ritorna l'oggetto di questo comparatore (siamo in una classe static) perche' puo' accedere ad un attributo del metodo della classe contenitore e' visibile ed usabile nella classe anonima cosi' come sarebbe visibile ed usabile una variabile di questo metodo, la classe anonima vede comunque non solo gli attributi della classe che la contiene ma anche le variabili locali e gli argomenti della funzione che l'ha creata. C'e' un piccolo problema perche' chiamando `createByRankComparator` lo ritorna qualcuno e nonostante sia uscito dalla funzione qualcuno potra' usare i parametri locali della funzione. Sotto sotto quando creo questo `new Comparator` i parametri locali della funzione vengono copiati durante la costruzione dell'oggetto, ma se l'oggetto poi cambia la copia non cambia, quindi la condizione per fare questa roba e' che sia l'oggetto che la variabile locale siano `final`, in java `final` o `effective final` perche' in effetti a tutti gli effetti e' `final` per cui Java si accorge che nessuno nella funzione ne cambia il valore e quindi lo tratta a tutti gli effetti come `final`.

Qui c'è una funzione a cui passiamo un parametro e all'interno della funzione creiamo una classe anonima che poi usa il parametro della funzione.

Consiglia che quando abbiamo dei dubbi facciamo le classi statiche perché tanto è quasi sempre quello che vogliamo

Serve per verificare la correttezza del programma, ma e' una tecnica di verifica ottimistica, puo' accrescere la nostra fiducia nella correttezza ma non dimostrarla, se facendo test non trovo neanche un problema sono fiducioso che non ci siano problemi ma non vuol dire che non ci siano bug, non e' una prova di assenza di errori. C'e' una sorta di glossario utile per chiarire a cosa ci riferiamo:

- Malfunzionamento: dal punto di vista esterno e' chiaro che qualcosa non abbia funzionato.
- Difetto (fault): la motivazione dell'errore l'istruzione era sbagliata ed era un difetto del codice.
- Sbaglio (mistake): perche' ho messo quell'errore nel mio codice (ero ubriaco, colpa del linguaggio), dobbiamo capire le motivazioni vere perche' ci sono i malfunzionamenti.

La presenza di un malfunzionamento implica la presenza di un difetto mentre l'assenza di malfunzionamento non implica l'assenza di difetti. Attenzione che il difetto non sempre si manifesta subito con un malfunzionamento.

- Unit testing: strutturarle, lo faccio guardando il codice che voglio testare
- Functional testing: funzionale, lo faccio non sapendo com'e' stato scritto il codice che voglio testare

35 Struttura di un test

Ogni stimolazione (caso di test) dev'essere divisa in tre momenti:

- Set up: predisporre il contesto
- Exercise: stimolazione della funzionalita' che vogliamo testare
- Verifie: verificare che la funzionalita' abbia funzionato come vogliamo

Ci sono per ogni IDE delle librerie che mi aiutano a fare il test (assert true) che lavorano a livello di astrazione alto e mi permettono di controllare la veridiciata' di alcune cose, abbiamo una comodita' di scrittura di questa roba e la possibilita' di scriverne tanti di questi casi e posso distinguere quelli che voglio fare o no, ho tanti punti di ingresso del programma e' quindi un modo piu' flessibile ed efficiente piuttosto che fare tanti main diversi.

36 Qualita' Test

- Veloce: Il fatto di poter verificare man mano che si fanno le funzionlita' che funzionino e' utile farlo man mano. Se questa operazione di testing fosse lenta noi non lo faremmo. Non dev'essere un attivita' dispendiosa in termini di tempo, ci sono addirittura metodologie di programmazione per

cui scrivo prima i test che le funzioni. Fare continuamente i test di non regressione, se aggiungo qualcosa di nuovo devo verificare che le funzioni vecchie non vengano meno. Non e' possibile che tutti i test siano veloci, i test di unita' deve essere veloce.

- Ripetibile: ci serve poter ripetere il test per essere sicuro di correggere l'errore. Ci serve anche per scrivere le asserzioni perche' se devo verificare automaticamente il funzionamento a tempo di scrittura devo sapere i valori che mi ritorna. Non devono fallire qualche volta e qualche volta no, dobbiamo trovare dei test per cui se il codice e' sbagliato crashi sempre.
- Indipendente: se devo tirare in mezzo tutto il programma non mi da indicazioni su dove c'e' stato l'errore, deve quindi essere specifico. Ci devono essere poche connessioni con altre parti del codice.
- Centrato: vuol dire strettamente coeso, come nella modularizzazione la disaccoppiazione interna e la coesione interna.
- Leggibile: Quando metto un apparato di controllo se questo e' sufficientemente complesso devo controllare anche lui, se il codice con cui faccio il test e' piu' complesso di quello da testare il test e' inutile (perche' potrebbe esserci li' dentro un errore). Dev'essere da scrivere e da leggere. Per ottenere questa cosa devo avere funzioni di alto livello e scrivere bene il testo che comporta che le cose che abbiamo definito per la buona progettazione si riversano anche sul test. Se ci metto dei while o if nel test probabilmente c'e' qualcosa che non va, il testo dovrebbe testare qualcosa di lineare

37 Eccezioni

Il fatto che i metodi sollevano eccezioni devo essere in grado di testarlo, devo mettere qualcosa che mi dice che non e' possibile eseguire quella istruzione senza che ci sia un'asserzione, lo facciamo in un try and catch aggiungendo un fail, puo' avere anche un messaggio per specificare il tipo di eccezione che mi aspettavo

38 Encapsulation

Cosa testare: metodi provati e pubblici. Non e' una questione che ha un'opinione unica su cosa testare, il libro non e' cosi' contrario per testare i metodi privati, il prof invece dice che i metodi privati non andrebbero testati, il discorso e' che il test si dovrebbe porre lato cliente (testare solo quello che il cliente potrebbe fare), perche' i metodi privati non li testo direttamente ma se sono li' vuol dire che ci sono dei metodi chiamati dal cliente che andranno a testare quella funzionalita'. Perche' dice che e' meglio fare in questa maniera, perche' e' un problema di fragilita', se cambio dei parametri nei metodi privati (che sono privati e quindi come abbiamo detto possiamo modificare senza preoccuparci di quello che c'e' fuori) dobbiamo poi cambiare tutti i parametri dei test. Ma questo

problema ce lo abbiamo anche con gli attributi, e' molto comune che a fronte di una certa stimolazione non sia sufficiente verificare cosa mi e' stato ritornato per vedere se la funzione mi ha tornato quello che volevo, molto spesso le funzinoi modificano lo stato dell'oggetto. Il metodo con cui andiamo a ragionare:

- Non ho l'accesso diretto ma ho un metodo che me ne da una rappresentazione? Ad esempio la mia carta non ho direttamente l'accesso getRank ma ho un toString che me ne da una rappresentazione testuale che mi da una rappresentazione dello stato interno
- Aggiungere un metodo per dare l'accesso alle informazioni, e' una cosa brutta perche' andiamo a scrivere una parte di codice nella classe esclusivamente per questioni di testing, non perche' serve qualcosa al cliente. Un altro modo sarebbe allargare l'encapsulation.
- Creare dentro al test un metodo (helper) in grado di ricavare tale informazione o una sua derivata. La size di un deck noi non l'abbiamo, non ci dice quante carte ci sono ancora, in qualche test potrebbe interessarmi scoprire qual'e' la dimensione del mazzo, questa roba con la versione attuale di deck non e' fattibile, pero' non voglio aggiungere il metodo size. Abbiamo messo un metodo non efficiente che fa tante draw sul mazzo e conta quante ne ha fatto e dopo ricrea un mazzo con quelle che ha fatto, non e' una cosa efficiente ma senza volere modificare il codice della classe ma mi verifica la size del mazzo.
- Usiamo la reflection o introspezione: e' un argomento avanzato e non ce ne parla sul libro c'e' ma non la trattiamo e non la consideriamo parte del programma.

39 Test Coverage

Ci interessa dal punto di vista teorico. Un test non puo' dimostrare l'assenza di un errore ma non puo' dimostrarne l'assenza. Se e' solo una cosa di ottimismo (man mano che lo facci sono sempre piu' fiducioso) quand'e' che mi posso fermare?

```
class Trivial {
    static int sum(int a, int b) {
        return a+ b;
    }
}
```

Il discorso e' simili al discorso fatto su stati astratti e concerti, per vedere che un pezzo di codice sia corretto dovrei fare un test esaustivo (provare tutti gli input) perche' non posso dare per assodato che se vale per determinati input per altri (tipo numeri piu' grandi) vada bene lo stesso. Nell'esempio scopriamo che il dominio di input e' enorme nonostante dobbiamo solo testare due numeri e ci metteremmo 600 anni, inoltre chi mi dice qual'e' il risultato corretto?

devo avere qualcuno che mi scriva tutti i risultati corretti per ogni operazione. Quindi non posso pensare di testare tutti i possibili casi. Allora devo trovare dei ragionamenti che mi dicono che tot test sono sufficienti.

39.1 Testing coverage

Definito delle caratteristiche del codice che posso andare a coprire, quali sono le cose che voglio verificare, perche' affinche' un difetto venga evidenziato e si traduca in un malfunzionamento cosa deve capitare?

- Io esegua l'istruzione in cui e' presente il difetto, se durante il test non eseguo mai l'istruzione che ha il problema, qualunque test io faccia non ho speranze di catturare il malfunzionamento

Questo vuol dire che se riusciamo a tenere traccia durante l'esecuzione dei test i comandi che sono stati eseguite alla fine dell'attivita' di testing possiamo sapere quali righe sono state eseguite, ad esempio un test che non copre tutti i comandi e' un test che possiamo giudicare insufficiente. Su IntelliJ c'e' la possibilita' di testare i comandi, mi fa eseguire il programma con run test with coverage (il run con lo scudo) riesegue i test e alla fine mi presenta le informazioni di quello che e' stata la visione a livello di classi in percentuale quanto vengono coperte dai test. Ci sono moltissimi criteri di copertura:

- Strutturali(che identificano una qualche caratteristica del codice)
- Funzionali (che identificano una qualche caratteristica delle specifiche)

39.2 Stubs

Test Double e' un termine generico per un qualunque tipo di oggetto che viene messo al posto di quello reale al fine di permettere un attivita' di testing (in inglese vuol dire controfigura). Sono funzioni che hanno meno funzionalita' di quelle che testo, non esiste ancora la parte di codice che fa quello che voglio, sostituiamo questa parte con un oggetto fittizio che mi ritorna quello che voglio ma che in realta' non implementa la funzione che vogliamo. In questa maniera ho due vantaggi:

- Posso cominciare a testare il codice senza aspettare i moduli da cui dipendo
- Abbiamo parlato di isolamento, se per caso l'errore non fosse nel mio codice ma venisse da un codice da cui dipendo sarebbe un casino, se invece il codice da cui dipendo e' sostituito da un risultato che so essere giusto, so che l'errore dipende per forza dalla mia parte di codice.

Ci servirebbe usare stub quando:

- L'oggetto non esiste ancora o non e' facilmente accessibile
- L'oggetto fornirebbe dei dati non deterministici/non prevedibili: il tempo corrente , la temperatura corrente

- Può presentare delle situazioni difficilmente riproducibili (errore di trasmissione, esaurimento della memoria)
- Quando ho che il sistema reale è lento, vado a chiedere ad un database e non voglio aspettare che pensi, voglio la risposta immediata

La composizione e' un concetto fondamentale della progettazione, ne abbiamo cominciato a parlare con il diagramma delle classi (la freccia con il rombo nero), il libro parla del concetto piu' generale, il modo di comporre le varie astrazioni in una maniera significativa per ottenerne di piu' complesse, il divide et impera domina ancora. Comporre diverse astrazioni non vuol dire farle bene, dobbiamo cercare di farlo bene. Robillard distingue due tipi di composizioni (scopi):

- Composizione in senso proprio: uso la composizione per rappresentare il mio oggetto, ad esempio il deck e' creato come composizione di carte, lo stato dell'oggetto deck e' caratterizzato dalle carte che lo compongono. Fa parte della definizione dello stato dell'oggetto quello che e' insieme degli oggetti che lo compongono.
- Aggregazione (composizione con scopo di delega): compongo e contengo all'interno degli altri oggetti ma loro non fanno parte del mio stato non mi definisco, ma mi serve conoscerli per poter svolgere altri compiti.

Questi termini escono un po' dalla terminologia standard, non e' la terminologia che si usa in UML. Il concetto di composizione e' talmente importante che e' usato all'interno dei un pattern.

40 COMPOSITE PATTERN

In cui abbiamo che la situazione che vogliamo andare a gestire e' avere gruppi di oggetti vedibili che si comportano come un solo oggetto. Voglio avere un modo per cui il mio cliente si rivolge ad un oggetto o ad i composti senza sapere qual'e' la differenza, si fa tramite un'interfaccia elemento che mi gestisce tutte le operazioni che posso fare, succede che posso avere un gruppo di gruppi. Il sistema dev'essere capace di gestire i gruppi senza saperlo, come se fossero elementi base, perche' hanno la definizione delle stesse funzioni.

Devo implementare i metodi usando i mazzi componendo il metodo isEmpty partendo dai metodi isEmpty delle mie componenti.

Card source composite ha semplicemente preso elemento per elemento di quelli che gli abbiamo passato copiando il riferimento e mettendoli in una nuova lista, questa cosa andrebbe bene se abbiamo degli oggetti immutabili, ma i mazzi non lo sono. Quindi nel costruttore c'e' un escaping reference. e' stata fatta questa cosa perche' evitare escaping reference e' difficile e va contro l'efficienza, magari posso ottenere risultato simili costruendo delle classi che incapsulano la non sicurezza. Questa e' una classe pubblica, ma potrei dire che e' una classe di passaggio creando un'altra classe che maschera questa cosa e limitarne alcuni utilizzi, tipo creando una classe MultiDeck in cui passo solamente il numero di mazzi che voglio e gli faccio creare dentro i mazzi, cosi' rimangono il segreto di quella classe. Così facendo non sto esportando al cliente l'escaping reference nonostante ci sia. E' un approccio che si chiama composizione per delega. Generalmente si riconosce che per chi studia l'obj orientation si pensa che l'ereditarietà sia più avanzata moderna e preferibile, e può essere che si usa troppo.

questo esempio di Composite è un primo esempio in cui ci accorgiamo che questa lettura del codice non è esplicitata e così ben comprensibile, la parte statica di definizione di `isEmpty` o `draw` sono non rappresentative di quello che è il comportamento dinamico bisogna quindi usare un nuovo diagramma: sequenza.

Questo diagramma ha una scatola per ogni oggetto di cui andremo a parlare, quando il client chiama `isEmpty` su quello che lui conosce (`s1`) com'è che viene svolta questa funzione, in realtà questo messaggio va ad iterare sugli oggetti che compongono la classe composta, per cui sul primo deck e sul secondo composite Card source. Sto dando l'ordine di esecuzione di una possibile esecuzione, mi dà il segnale delle funzino che chiamo sui vari oggetti e mi dice quali sono i valori ritornati, ogni colonna rappresenta un oggetto ed ogni freccia un'invocazione del metodo, la lettura è dall'alto al basso. Le frecce tratteggiate in verticale sono la linea di vita dell'oggetto, quando diventa rettangolo significa che l'oggetto sta facendo qualcosa (è sullo stack dell'activation record, quindi è un oggetto attivo). Nell'esempio di iterator abbiamo visto che l'uml non obbliga a mettere le frecce di ritorno (come nel metodo `hasNext()`), esplicitandolo abbiamo ritornato presente, poi mettiamo nella chiamata di `nextCard` mettiamo il controllo `[presente]`, `if presente == true` chiama `next`, se è falso vado a vedere cosa c'è dopo. In uml per fare iterare le cose metto gli asterischi nelle box. Potrei voler fare due diagrammi diversi per dover rendere più leggibili i diagrammi con diverse situazioni.

I commenti sono pericolosi (idea delle metodologie agile), il commento lo faccio in maniera esecutiva tramite i test.

Il prossimo pattern che vedremo assomiglia al composite con la differenza che l'oggetto che aggrega aggrega un unico oggetto di interfaccia, si mette come contenitore di quello che va a contenere, per indicare che uno stesso schema UML può essere usato per cose completamente diverse. Pattern Decorator: Aggiungere dinamicamente delle funzionalità a degli oggetti. Ad esempio vogliamo aggiungere il fatto che a ogni invocazione di `draw` venga scritta su console la carta estratta. La soluzione semplice ma da evitare è quella di creare una classe per ogni possibile decorazione (ogni possibile caratteristica della classe che voglio). Una seconda soluzione semplice ma che è un antipattern è la god class, creo una sola classe in cui tramite attributi booleani e switch attualizzo le varie decorazioni. Non va bene perché è una classe grossa, potrei doverci mettere un sacco di condizioni e magari non ne uso nessuno, al suo interno memorizza tanti attributi false che non uso mai (ho oggetti grossi anche in casi semplici). 2 svantaggio: devo mettere una decorazione, cosa devo fare? devo aggiungere un attributo alla classe e con i vari metodi calcolare il costo della pizza ad esempio se quel `if` è verificato, ma è una cosa che va a violare l'open close principle (per apportare delle modifiche non devo mettere mano al codice già scritto). Mi piacerebbe avere il concetto della singola decorazione separato e mettere in maniera coesa nella classe che decorerà in quella maniera ha degli effetti (tutto raggruppato e non sparpagliato in giro). Ricordiamo che i pattern sono buone soluzioni in cui sono privilegiate alcune qualità invece che altre, quelli che stiamo vedendo risultano semplici ma potrebbero essere più complessi nell'interazione

tra le classi, l'interazione però viene standardizzata quindi mi reste da guardare com'è stata implementata nelle singole classi, la cosa interessante dei pattern è che spesso si sovrappongono tipo decorator e composite, posso avere la card source composita e le decorazioni. Questa cosa funziona bene fintanto che le decorazioni siano indipendenti tra di loro e combinabili. Il pattern decorator è usato in java nello stream, perchè ogni volta creo uno stream che ha delle caratteristiche uguali a quelle di prima ma a cui aggiungo altre funzionalità

41 Law of Demeter

E' una cosa di cui abbiamo già parlato in diverse maniere, legato al principio di disaccoppiamento dei moduli (con pochi dipendenze tra loro) ed evitare i getter. Bisogna nascondere il piu' possibile (avere conoscenza di meno cose possibile) ed astrarre le funzionalita'. Ogni unita' di programma dovrebbe interagire solo con le unita' che conosce direttamente, non parlare agli sconosciuti e tell don't ask (di quello che vuoi venga fatto piuttosto che estrarre le informazione e poi lavorarci). Viene trattato dopo la composizione perche' ora sappiamo che all'interno di un oggetto possono esserci cose che vanno a caratterizzare altri oggetti si capisce che questa cosa diventa una catena di inclusioni, nel nostro esempio il gioco di carte ha le foundation che sono composte da pile che sono composte da carte, notiamo la composizione in successione. Quando voglio prendere la carta dalla lista di carte finali parto dal gameModel, un modo di pensare di fare questa operazione e' dare le foundations tramite il suo metodo getPile mi restituisce una pila che a sua volta con una getCard mi da la carta. Con questo meccanismo io client chiedo man mano di portare fuori i segreti di amici e farli miei per usarli direttamente. Non e' che io gameModel conoscevo foundation e gli altri nascosti, in qualche modo li tiro fuori di volta in volta. La cosa invece bella e' fare la delega: io dico alla foundation che ho bisogno di aggiungere una carta ad un giocatore e gli fornisco le cose che gli servono per fare le operazioni necessarie tramite ad esempio il metodo addCard in foundation tramite cui aggiunge la carta senza esportare l'informazione al client. Il risultato e' che i vari oggetti dialogano solo con i loro successori non con gli amici degli amici, le conoscenze indirette non vengono sfruttate, la regola e' di andare a gestire questa cosa aggiungendo dei metodi specifici. C'e' una cosa che avevamo visto parlando dell'antipattern temporary field (aggiungere troppi metodi) ma in questo caso e' la cosa giusta passare i parametri tramite metodi per isolare i segreti. Le regole con cui controllare il rispetto di questo pattern non sono facili, ci sono dei tentativi che pero' e' molto facile da bypassare questo controllo:

- Evitare i getter, per non esportare il segreto.

Il codice di un metodo dovrebbe accedere solamente a:

- This cioe' a se stesso
- Agli argomenti passati al metodo, se ho dei parametri sono cose che e' previsto su cui lavori

- Agli oggetti che creo durante l'esecuzione del metodo, le variabili locali a cui assegno qualcosa che creo
- (se proprio e' necessario) oggetti globali, ad esempio il Singleton posso raggiungerlo in qualunque punto del programma o campi statici di altre classi

Se c'e' qualche altra cosa che il metodo sta usando qualcos'altro drizzare le orecchie, notiamo che qui mancano anche i campi privati di altri oggetti della classe ma e' una cosa che molto spesso accetteremo. Ci sono certe situazioni in cui sapendolo viene comodo andare contro questo pattern. Per esportare le informazioni fornisco dei metodi che danno la soluzione pero' non esportano i dati. Nelle soluzioni prima proviamo con i getter perche' sono il metodo piu' semplice per risolvere il problema ma poi cerchiamo sempre il modo per togliere i getter.

42 Polimorfismo con Interfacce

Le cose piu' difficili da capire dell'Inheritance sono il polimorfismo ed il collegamento dinamico. Ad un identificatore di una classe puo' sempre essere associato un oggetto che eredita la classe, l'ereditarieta' e' il meccanismo di Java per realizzare la generalizzazione UML. Implementazione: (freccia tratteggiata) era prendersi dei doveri, sobbarcarsi l'onere di implementare i metodi dichiarati nell'interfaccia per poter essere visti come qualcosa di piu' astratto rispetto a se stessi, non abbiamo mai fatto dei metodi di default nelle interfacce perche' ad esempio sono limitati a fattorizzare parti di codice che non dipendono dallo stato.

43 Polimorfismo con Classi

Quando vado a spostarmi sulla classe (una classe estende un'altra classe) questa cosa cambia radicalmente, perche' la classe da cui eredito non ha i limiti dell'interfaccia, non vuol dire solamente che ad esempio un `CircularDeck` puo' essere trattato come un `Deck` ed essere passato come tale quando viene richiesto (posso essere citato in varie maniere), ma a differenza dell'interfaccia eredito anche gli attributi ed i metodi specifici del `Deck` (la classe da cui vengo esteso), mi permette di definire una classe dalla differenza tra quella ed una classe gia' esistente, mi permette di non partire da zero. Eredito ma non sono costretto, poteri fare l'override del metodo che mi viene fornito Perche' allora se e' cosi' bello fare l'ereditarieta' rispetto alle interfacce perche' non abbiamo solo loro? Il fatto di avere questa ereditarieta' complica un altro discorso, perche' potrei avere una struttura a grafo ed avere un problema di capire da chi ho ereditato i metodi ed anche diventa difficile nel caso di override qual'e' l'implementazione che prevale.

Albero di ereditarieta' in Java:

- Non supporta ereditarieta' multipla
- Ogni classe al massimo un padre, ma puo' avere nonni, bisnonni etc. Ereditato sequenzialmente da piu' classi non ho quindi ambiguita' delle cose che eredito

La caratteristica di Java e' che tutte le classi ereditano da una classe: `Object`. Anche se non dico nulla eredita sempre da `Object` abbiamo gia' visto che tutti i metodi di `Object` (`equals`, `toString`) possono essere usati da qualunque oggetto.

43.1 Ereditarieta' degli attributi

Ereditiamo l'implementazione di un metodo, quello che e' piu' nuovo quando parliamo di classi e' che ereditiamo anche gli attributi (nelle interfacce non posso ovviamente ereditarle, si intendono variabili legate all'istanza quelle che abbiamo sempre considerato i segreti della nostra classe). Quello che facciamo e' che ereditiamo gli attributi e li ereditiamo tutti:

- non posso cambiarne il tipo, posso pero' ovviamente assegnarci diversi valori
- se sono private non posso accederci dai metodi della sottoclasse:
 - esiste un nuovo ambito di visibilita' protected: possono vederlo tutte le classi anche fuori dal package (package e' il livello di protezione a livello di package, non bisogna esplicitarlo) se dicono che mi stanno estendendo (sono mie figlie)

non posso cancellarli. Eredito anche gli attributi privati, solo che non posso accedervi se non tramite i metodi che eredito e che li usano

43.2 Ereditare metodi

E' leggermente diverso, abbiamo gia' esperienza con le interfacce e facevamo gia' override ma era strano perche' non sovrascrivevamo niente, ma stava a significare che anche se ci fosse stata una definizione di come veniva fatto quel metodo ora la sovrascrivevo, vale anche per un metodo che eredito. Mi permette di ridefinire in maniera piu' appropriata delle caratteristiche che nella classe padre non esistevano, oppure ad esempio il toString deve cambiare (potrei anche fargli stampare quello che avrebbe stampato il padre e poi fargli stampare le mie cose), diventa ancora piu' chiara il collegamento dinamico. Quando faccio l'override non e' lecito sostituire quello che faceva la classe padre con qualsiasi cosa. principio di liskov: **Quando sovrascriviamo una cosa gia' fatta deve rispettare certe condizioni, esprimibili con pre e post condizioni.** Una cosa che non rispettera' nessun contratto e' fare l'override e svuotare la funzione (graffa aperta e vuota), questa roba non va assolutamente bene e non va fatta.

In UML se faccio overriding devo riscrivere il nome. Nel codice il compilatore non contesta se non scriviamo l'override ma se la scriviamo ci aiuta a controllare se effettivamente stiamo facendo l'override di qualcosa.

Attenzione a non confondere con l'overloading: posso dare lo stesso nome a vari metodi a patto di cambiare i parametri. Basta cambiare il tipo o il numero dei parametri per aver eun nuovo metodo indipendentemente dal fatto che abbia lo stesso nome. Cambiare invece il tipo ritornato a quello di un sottotipo non crea un nuovo metodo.

43.3 Sbtyping: precisazioni

Posso evocare l'implementazione dei metodi che sovrascrivo tramite `super.Metodo()`.

Cosa succede quando creo un istanza di tipo Figlio e l'assegno ad un identificatore di tipo Genitore? Chiamando i metodi su questo oggetto anche se il Genitore ha lo stesso metodo a run time guarda se Figlio ha ridefinito quel metodo usa il metodo del Figlio. Attenzione che non esiste un oggetto Genitore in questo caso ma e' sempre un oggetto Figlio con un identificatore Genitore. Posso usare un attributo di Figlio se lo assegno a Genitore come in questo caso?

Dipende, quando io so che dentro quell'oggetto c'è in realtà un figlio anche se lo sto trattando come padre posso fare un down cast dicendo di guardare l'identificatore padre come figlio. Tiene traccia che l'identificatore dell'oggetto è cambiato ma a run time sa che in realtà è un oggetto di tipo Figlio.

44 Adapter Pattern

Ci permette di ragionare sulle differenze tra composizione ed ereditarietà. Spesso quando dovremo modellare uno schema potremmo scegliere tra le due composizioni. È un pattern che mira a risolvere il problema di rendere compatibili due cose che non lo sono (tipo la spina elettrica), magari ho trovato un componente in una libreria che offre una funzionalità che vorrei inserire nel sistema però il modo di invocarla e lavorarci sono incompatibili con quella che era la mia idea. Ho la funzionalità ma non ho la presentazione (nome funzioni, parametri etc) che mi serve, quello che devo fare è inserire un adattatore che implementa l'interfaccia della classe che vogliamo e parla con l'interfaccia della funzione che vogliamo inserire. Può succedere quando faccio grossi upgrade, ci sono classi che vogliono vedere funzionalità nuove nella nuova maniera ed alcune nella vecchia. Come posso pensare di costruire questo adapter? In qualche modo dobbiamo fornire una nuova astrazione, dobbiamo fare in modo che il cliente dialoghi con un'interfaccia che lui ritiene quella corretta ma è lei che a sua volta va a prendere i metodi corretti da qualche altra parte.

44.1 Class Adapter

Adatto l'intera classe e l'adatto tramite estensione (generalizzazione, ereditarietà), l'adapter eredita da adaptee, a questo punto adapter ha un metodo request e oldRequest, se il cliente invoca il metodo oldRequest fa esattamente le stesse cose che faceva prima. Mentre voglio che se il cliente tramite adapter chiami request e che venga ritornato oldRequest, basta dentro adapter chiamare oldRequest:

```
public class Adapter extends Adaptee implements Target {
//È un override dell'interfaccia
    @Override
    public void request() {
        this.oldRequest();
    }
}
```

Per cui in qualche modo sto gestendo un caso in cui posso avere un singolo oggetto vedibile in due maniere diverse ed il codice che ho dovuto scrivere è solo quello di rimappare la nuova interfaccia sulla vecchia.

44.2 Object Adapter

In questo caso adapter e' composto da adaptee, uno dei miei campi sara' un adaptee. Sono due modi di adattare, uno adatta una classe, l'altro un oggetto.

```
public class Adapter implements Target {
    private final Adaptee adaptee;

    public Adapter (Adaptee adaptee) {
        assert adaptee != null;
        this.adaptee = adaptee;
    }

    @Override
    public void request() {
        adaptee.oldRequest();
    }
}
```

Ho due oggetti che metto in relazione, quando devo chiamare request su adapter invoco il metodo sull'attributo che ho creato. Ovviamente in questo caso non potrei chiamare oldRequest su adapter, perche' non lo implementa.

Non sempre posso fare l'ereditarieta' della classe, se eredito gia' da un'altra classe sono fregato. Pensiamo invece adaptee come una classe da cui ereditano altre classi (oppure che sia un'interfaccia), ha senso adattare un'interfaccia? solo nel secondo caso perche' il secondo caso puo' adattarsi anche ad altre sottoclassi. Fa capire che uno stesso compito puo' essere svolto in due modi diversi, ed anche le conseguenze sono diverse.

45 Command Pattern

E' un pattern che incapsula una funzionalita', e' simile al pattern strategy, nel quale le funzioni erano interscambiabili (ad esempio ordinare in modi diversi, o giustificare il testo) attuandoli con metodi diversi. Nel pattern Command possiamo anche avere funzioni completamente diversi, ed inoltre vengono aggiunti dei metodi che non implementano le funzioni ma sono dei ragionamenti su quello che fa, nell'esempio la funzione e' execute ed il metodo che ci ragiona e' l'undo(). Per cui ho un'interfaccia astratta che mi dice che qualunque oggetto che aderisce deve implementare quei metodi e poi ho l'effettiva implementazione nelle classi concrete.

Ci sono alcuni elementi da decidere:

- l'oggetto su cui opera, come rendo disponibile al comando il contesto in cui opera? fa parte del suo stato (setter o costruttore) o glielo passo come parametro?
- Ritorna un valore? Da un feedback ho e' una procedura?

- Cosa succede alle precondizioni? chi e' responsabile del loro soddisfacimento?

Mi piacerebbe fattorizzare una parte comune tra tutte le mosse ma non posso farlo se una mossa e' la generalizzazione delle altre, Java ci permette di costruire una generalizzazione ad hoc. Definiamo una classe che non e' istanziabile (non possiamo creare metodi da quella classe)

46 Classi astratte

E' tra classe ed interfaccia, e' una classe di cui posso avere attributi ed implementazioni e posso anche non avere alcuni metodi, come se fosse un oggetto incompleto. Quando lavorando su un'astrazione per definirla bene dobbiamo definire degli attributi non usiamo l'interfaccia ma una classe astratta, in qualche modo se ha senso e' sempre utile mettere un'astrazione davanti alla classe concreta. Ereditano sempre da solo una classe. Utili per fattorizzare parti di codice e responsabilita', attenzione che continua a valere divieto di eredita' multipla. In UML il nome di una classe astratta viene scritto in corsivo. Ha senso parlare di costruttore di una classe astratta? Ci serve avere un costruttore, perche' probabilmente nella classe astratta ho degli attributi o conoscenze che devono essere istanziate. Il costruttore viene chiamato implicitamente (o esplicitamente) dai costruttori delle classi figlie.

C'e' un modo sintattico per evitare di creare sottoclassi, mettendo final davanti alla classe (questa classe non puo' essere ulteriormente specializzata).

Tornando a Command, potrei creare una mossa astratta DefaultMove che fattorizza le parti comuni di DeisrdrMove RevealTopMove e CardMove.

C'e' un problema da risolvere? l'attributo (se e' privato) come viene visto dalle sottoclassi?

```
public abstract class AbstractDecorator implements CardSource {
    private final CardSource element;

    public AbstractDecorator (CardSource element) {
        assert element != null;
        this.element = element;
    }

    @Override
    public Card draw(){
        return element.draw();
    }

    @Override
    public boolean isEmpty(){
        return element.isEmpty();
    }
}
```

```

}

public class LogginDecorator extends AbstractDecorator {
    public LogginDecorator(CardSource element) {
        super(element);
    }

    @Override
    public Card draw() {
        Card card = super.draw();
        System.out.println(card);
        return card;
    }
}

```

In questo modello abbiamo `super.draw` sono la stessa cosa, e per ogni classe che decora devo scrivere queste tre righe, riusciamo a strutturarlo in modo piu' raffinato in modo che ogni volta che devo aggiungere un nuovo decoratore dico qual'e' la differenza dagli altri, posso farlo con il polimorfismo.

Sposto questo schema nella classe astratta:

```

@Override
public Card draw() {
    Card card = element.draw();
    specificDecorationAction(card);
    return card;
}
abstract protected void specificDecorationAction(Card card);

```

E nella sottoclasse dico semplicemente:

```

public class LoggingDecoratore extends AbstractDecorator{
    public LoggingDecorator(CardSource element){
        super(element);
    }

    @Override
    protected void specificDecorationAction(Card card){
        System.out.println(card);
    }
}

```

47 Template

Quando esiste uno schema fattorizzabile a livello di una classe

48 Principi SOLID

- Singola responsabilit : Ogni classe dovrebbe avere una ed una sola responsabilit , interamente incapsulata al suo interno, un solo motivo per dover cambiare
- Aperto/Chiuso: Se devo estendere delle cose non lo devo fare modificando classi gi  esistenti ma trovando un'altra maniera (ereditariet , decorator...). Modificando le cose esistenti non avendo il controllo su chi ha usato quelle cose come base di partenza del loro codice gli aggiungo funzionalit  che non sanno di avere. Un'entit  software dovrebbe essere aperta alle estensioni, ma chiusa alle modifiche.
- sostituzione di Liskov: Gli oggetti dovrebbero poter essere sostituiti con dei loro sottotipi, senza alterare il comportamento del programma che li utilizza. E' vero che l'ereditariet  crea una relazione di sottotipo. Se l'oggetto figlio non garantisce le stesse qualit  che garantisce il padre non va bene. Questo si rimappa molto bene sul discorso delle precondizioni
- Interface segregation: meglio fare tante interfacce specifiche per usare il giusto livello di astrazione quando ci serve
- Inversione delle dipendenze: una classe dovrebbe dipendere dalle astrazioni, non da classi concrete

48.1 Liskov

Se ridefinisco un metodo, la nuova implementazione:

- Non puo' avere precondizioni piu' strette: devo andare ad aggiungere casi da tagliare. Tutte le cose che ho esplicitato prima come l'assert le mantengo quando eredito la classe. Ricordarsi che quando faccio l'override perdo gli assert.
- Non puo' avere postcondizioni piu' larghe
- Non puo' richiedere parametri piu' specifici: e' l'overloading, so fare qualcosa in piu' ma non mi dimentico quello che eredito
- Non po' ridurre la visibilit  dei metodi
- Non puo' lanciare nuovi tipi di eccezioni
- Non puo' avere un tipo di ritorno meno specifico

Di queste 6 cose quelle che ci interessano di piu' sono le prime due, gli ultimi tre ci danno errori a compilazione.

49 Pattern Observer

Fattorizziamo in un punto unico lo stato (che e' la parte comune di informazione, lo chiamiamo modello o subject) e mettiamolo in un oggetto a parte (Subject) che verra' osservato dagli altri (Observer).

In Java c'erano delle classi consigliate per realizzare questo pattern:

- interfaccia `java.util.Observer`
- `java.util.Observable`

49.1 Observer

Come colleghiamo Observable e Observer? Per essere sicuri che le viste (Observer) non si devo conoscere tra loro, inoltre vogliamo minimizzare la conoscenza tra il subject (oggetti osservabili) e le viste. Questa conoscenza e' mappata dall'aggregazione con una direzione: l'oggetto osservato conosce gli osservatori, ha al suo interno delle informazioni degli observer. Abbiamo due modalita' di implementare l'observer:

- Pull: comunico che qualcosa nello stato e' cambiato e ogni observer puo' andare a guardare cos'e' cambiato, e' l'observer che viene a tirare fuori l'informazione
- Push: io observer ti butto fuori le informazioni che ho al mio interno

Siccome l'observer non conosce l'observable, in modalita' pull devo anche ritornare chi sono. Quali sono le motivazioni alla base dei due approcci: con il metodo push dimezzo il numero di chiamate, potrebbe essere che non sappiamo l'observer cosa vuole, se i dati che dobbiamo passare sono tanti non e' detto che sia conveniente per tutti gli osservatori passarglieli tutti, non e' detto che tutti gli osservatori siano interessati a tutto il contenuto del modello.

49.2 Observer Push

L'observer manda lo stato, il fatto che devo fare il casting

```
//OBSERVABLE
@Override
public void notifyObservers(){
    for(Observer observer : observers) {
        observer.update(null, state);
    }
};

//OBSERVER
@Override
public void update(Observable model, Object state) {
    if(state instanceof Integer) {
```

```

        doSomethingOn((Integer) state);
    }
}

```

49.3 Modalita' Pull

```

//OBSERVABLE
@Override
public void notifyObservers(){
    for(Observer observer : observers) {
        observer.update(null, state);
    }
};

//OBSERVER
@Override
public void update(Observable model, Object state) {
    if(model instanceof ConcreteObservable) {
        doSomethingOn(((ConcreteObservable) model).getState());
    }
}

```

50 Factory Method

e' il metodo di una classe che serve a creare un istanza ma non e' un costruttore . Definisce una interfaccia per creare un oggetto, ma delegando la scelta dell'oggetto stesso a sottoclassi. Potrebbero esserci occasioni in cui volendo costruire un oggetto e vorrei fare una new di qualcosa che gli diro' dopo, non posso fare pero' il collegamento dinamico con la new (non posso avere un costruttore virtuale). Allora quello che viene in mente di fare e' inglobare il costruttore dentro un metodo normale